Avaliação Técnica

Desenvolvedor Front End — Angular

Avalia conhecimentos práticos em Angular 17+, RxJS, NgRx/Signals, Angular Material, testes unitários, TypeScript e integração com APIs REST.

⏱ **Prazo de entrega**	📦 **Entrega**
4 dias corridos após recebimento	Encaminhar este documento com as perguntas respondidas e com o link do código público em sua conta do GitHub

1. TypeScript e Qualidade de Código

1.1. Refatoração

Considerando seus conhecimentos de TypeScript, qualidade de código e boas práticas, quais melhorias você faria no seguinte código:

```
class Produto {
 id: any;
 descricao: any;
 quantidadeEstoque: any;

 constructor(id: any, descricao: any, quantidadeEstoque: any) {
   this.id = id;
   this.descricao = descricao;
   this.quantidadeEstoque = quantidadeEstoque;
 }
}

class Verdureira {
 produtos: any;

 constructor() {
   this.produtos = [
     new Produto(1, 'Maçã', 20),
     new Produto(2, 'Laranja', 0),
     new Produto(3, 'Limão', 20)
   ];
 }

 getDescricaoProduto(produtoId: any) {
   let produto;

   for (let index = 0; index < this.produtos.length; index++) {
     if (this.produtos[index].id == produtoId) {
       produto = this.produtos[index];
     }
   }

   return produto.id + ' - ' + produto.descricao + ' (' + produto.quantidadeEstoque + 'x)';
 }

 hasEstoqueProduto(produtoId: any) {
   let produto;

   for (let index = 0; index < this.produtos.length; index++) {
     if (this.produtos[index].id == produtoId) {
       produto = this.produtos[index];
     }
   }

   if (produto.quantidadeEstoque > 0) {
     return true;
   } else {
     return false;
   }
 }

}
```

1- Tiraria os any do código e adicionaria tipagem específica às variáveis, como number ou string. Isso evita que erros silenciosos passem pelo TypeScript.

2- Incluiria a tipagem explicitamente no retorno das funções (como : boolean ou : string). Isso deixa o contrato da função claro de imediato, sem que seja necessário investigar toda a lógica interna para saber o que ela devolve.

3- As duas funções repetem o mesmo loop for, eu criaria uma função privada para centralizar essa busca. Além disso, eu trocaria o for pelo método .find(), que é mais eficiente, ele para o processamento assim que encontra o primeiro item compatível, em vez de percorrer todo o array sem necessidade.

4- No método hasEstoqueProduto, eu usaria um retorno reduzido. Como a comparação quantidade > 0 já resulta em true ou false, não é necessário o uso de um if para decidir o retorno; a própria expressão já resolve isso.

5- Adicionar o modificador private às variáveis da classe e as acessava via get.

6- Antes de acessar qualquer propriedade do produto, eu verificaria se ele realmente foi encontrado. Caso a busca não traga nenhum resultado, eu retornaria false para garantir que a aplicação não tente acessar algo que não existe.

7- Removeria a concatenação com o operador + e passaria a usar template literals (com as crases). Isso deixa a montagem das mensagens mais limpa e fácil de ler.

```
class Produto {
 constructor(
   private _id: number,
   private _descricao: string,
   private _quantidadeEstoque: number,
 ) {}

 get id(): number {
   return this._id;
 }
 get descricao(): string {
   return this._descricao;
 }
 get quantidadeEstoque(): number {
   return this._quantidadeEstoque;
 }
}

class Verdureira {
 private produtos: Produto[];

 constructor() {
   this.produtos = [
     new Produto(1, "Maçã", 20),
     new Produto(2, "Laranja", 0),
     new Produto(3, "Limão", 20),
   ];
 }

 private buscarProduto(productId: number): Produto | undefined {
   return this.produtos.find((produto) => produto.id === productId);
 }

 getDescricaoProduto(produtoId: number): string {
   let produto = this.buscarProduto(produtoId);

   if (!produto) {
     return "Produto não encontrado";
   }

   return `${produto.id} - ${produto.descricao} (${produto.quantidadeEstoque}x)`;
 }

 hasEstoqueProduto(produtoId: number): boolean {
```

```
  const produto = this.buscarProduto(produtoId);

  if (!produto) {
    return false;
  }

  return produto.quantidadeEstoque > 0;
 }
}
```

1.2. Generics e tipos utilitários

Implemente uma função genérica filtrarEPaginar<T> que recebe um array, um predicado de filtro e parâmetros de paginação (página e tamanho). A função deve retornar os itens da página atual e o total de registros filtrados. Use tipagem completa — sem any.
Exemplo de parâmetros:

```
 filtrarEPaginar<T>(
  data: T[],
  filterFn: (item: T) => boolean,
  params: PaginaParams
): Pagina<T>
```

Demonstre o uso com um exemplo concreto (ex: array de usuários com filtro por nome).

1- Eu comecei definindo as interfaces PaginaParams (para os dados de entrada) e Pagina<T> (para a resposta). Elas funcionam como os alicerces do código, garantindo que a paginação seja previsível e tipada, eliminando qualquer necessidade de usar o any.

2- A função filtrarEPaginar<T> foi desenhada para aceitar qualquer tipo de dado. O parâmetro filterFn recebe um predicado de filtro compatível com o método nativo .filter() do JavaScript.

3- como as interfaces de usuário costumam começar na página 1, eu usei a conta (pagina - 1) * tamanho para localizar o ponto de partida exato. Depois, o método .slice() entra em cena para extrair apenas aquela 'fatia' específica da nossa lista já filtrada.

```
interface PaginaParams {
 pagina: number;
 tamanho: number;
}

interface Pagina<T> {
 items: T[];
 total: number;
}

function filtrarEPaginar<T>(
 data: T[],
 filterFn: (item: T) => boolean,
 params: PaginaParams,
): Pagina<T> {

 const dadosFiltrados = data.filter(filterFn);
 const total = dadosFiltrados.length;

 const inicio = (params.pagina - 1) * params.tamanho;
 const fim = inicio + params.tamanho;

 const items = dadosFiltrados.slice(inicio, fim);

 return { items, total };
}
// Exemplo de Uso
```

```
interface Usuario {
 id: number;
 nome: string;
 ativo: boolean;
}

const usuarios: Usuario[] = [
 { id: 1, nome: 'Ana Silva', ativo: true },
 { id: 2, nome: 'Carlos Souza', ativo: true },
 { id: 3, nome: 'Ana Clara', ativo: false },
 { id: 4, nome: 'Beto Gomes', ativo: true },
 { id: 5, nome: 'Mariana Silva', ativo: true }
];

const termoBusca = 'Ana';
const parametros: PaginaParams = { pagina: 1, tamanho: 2 };

const resultado = filtrarEPaginar<Usuario>(
 usuarios,
 (usuario) => usuario.nome.toLowerCase().includes(termoBusca.toLowerCase()),
 parametros
);

console.log(resultado);
```

2. Angular — Fundamentos e Reatividade

2.1. Change Detection e OnPush

O componente abaixo usa ChangeDetectionStrategy.OnPush, mas o nome não é exibido na tela. Identifique o problema, explique o motivo e proponha a correção — sem alterar a estratégia, sem modificar PessoaService e sem remover o setInterval.

```
import { ChangeDetectionStrategy, Component, Injectable, OnInit, OnDestroy } from '@angular/core';
import { of, Subscription } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable()
class PessoaService {
 /** @description Mock de uma busca em API com retorno em 0.5 segundos */
 buscarPorId(id: number) {
   return of({ id, nome: 'João' }).pipe(delay(500));
 }
}

@Component({
 selector: 'app-root',
 providers: [PessoaService],
 changeDetection: ChangeDetectionStrategy.OnPush,
 template: `<h1>{{ texto }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
 texto: string;
 contador = 0;
 subscriptionBuscarPessoa: Subscription;
 constructor(private readonly pessoaService: PessoaService) {}

 ngOnInit(): void {
   this.subscriptionBuscarPessoa = this.pessoaService.buscarPorId(1).subscribe((pessoa) => {
```

```
    this.texto = `Nome: ${pessoa.nome}`;
  });

  setInterval(() => this.contador++, 1000);
}

ngOnDestroy(): void {
  /** ... */
}
}
```

O problema ocorre porque a estratégia ChangeDetectionStrategy.OnPush desabilita a detecção de mudanças automática contínua para o componente. O OnPush instrui o Angular a atualizar a view apenas em cenários específicos (mudança de referência em @Input, eventos disparados na UI, ou uso do async pipe).

Como a variável texto está sendo mutada de forma assíncrona dentro de um subscribe (após o delay de 500ms), essa mutação ocorre fora do ciclo de detecção ativo. O Angular não é notificado da alteração da propriedade local e, consequentemente, não re-renderiza o template. O setInterval presente no código, embora acione o Zone.js globalmente, não força a atualização deste componente específico justamente por ele estar blindado pelo OnPush.

Para resolver isso sem abrir mão da performance do OnPush, eu faria a injeção do serviço ChangeDetectorRef. Assim, logo após atribuir o novo valor à variável texto dentro do subscribe, eu chamaria o método markForCheck(). É como 'tocar o sino' para o Angular, avisando explicitamente que o componente está com informações novas (está 'sujo') e precisa ser revisado obrigatoriamente no próximo ciclo de detecção.

```
import { ChangeDetectionStrategy, Component, Injectable, OnInit, OnDestroy, ChangeDetectorRef } from
'@angular/core';
import { of, Subscription } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable()
class PessoaService {
/** @description Mock de uma busca em API com retorno em 0.5 segundos */
buscarPorId(id: number) {
  return of({ id, nome: 'João' }).pipe(delay(500));
}
}

@Component({
selector: 'app-root',
providers: [PessoaService],
changeDetection: ChangeDetectionStrategy.OnPush,
template: `<h1>{{ texto }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
texto: string;
contador = 0;
subscriptionBuscarPessoa: Subscription;
constructor(private readonly pessoaService: PessoaService, private readonly cdr: ChangeDetectorRef) {}

ngOnInit(): void {
  this.subscriptionBuscarPessoa = this.pessoaService.buscarPorId(1).subscribe((pessoa) => {
    this.texto = `Nome: ${pessoa.nome}`;
    this.cdr.markForCheck();
  });

  setInterval(() => this.contador++, 1000);
}

ngOnDestroy(): void {
  if (this.subscriptionBuscarPessoa) {
```

```
    this.subscriptionBuscarPessoa.unsubscribe();
  }
}
}
```

2.2. RxJS — eliminando subscriptions aninhadas

Refatore o código abaixo eliminando o subscribe dentro de subscribe. Use operadores RxJS adequados, evite memory leaks e explique brevemente sua escolha de operador:

```
ngOnInit(): void {
  const pessoaId = 1;

  this.pessoaService.buscarPorId(pessoaId).subscribe(pessoa => {
    this.pessoaService.buscarQuantidadeFamiliares(pessoaId).subscribe(qtd => {
      this.texto = `Nome: ${pessoa.nome} | familiares: ${qtd}`;
    });
  });
}
```

1- Eu escolhi o forkJoin para ganharmos performance. Como as duas requisições são independentes, não faria sentido esperar uma acabar para só então começar a outra. Com ele, as chamadas disparam juntas, em paralelo, e o código só segue adiante quando ambas retornarem com sucesso. É muito mais rápido e eficiente!

2- Para garantir a saúde do componente, eu usei o takeUntilDestroyed, que é o novo do Angular 16+. Ele limpa a inscrição automaticamente assim que o componente é destruído. Isso nos livra de toda aquela burocracia de gerenciar Subscriptions manualmente e evita vazamentos de memória (memory leaks).

3- Meu objetivo aqui foi deixar o código linear e declarativo. Ao usar os operadores do RxJS, a gente foge daquele aninhamento excessivo de funções, que torna o código difícil de ler. Além de ficar mais elegante, essa estrutura é fácil de evoluir: se amanhã uma chamada passar a depender da outra, basta 'plugar' um switchMap e a lógica continua fluida e organizada.

```
private destroyRef = inject(DestroyRef);
ngOnInit(): void {
 const pessoaId = 1;
 forkJoin({
   pessoa: this.pessoaService.buscarPorId(pessoaId),
   qtd: this.pessoaService.buscarQuantidadeFamiliares(pessoaId)
 })
 .pipe(takeUntilDestroyed(this.destroyRef))
 .subscribe(({ pessoa, qtd }) => {
   this.texto = `Nome: ${pessoa.nome} | familiares: ${qtd}`;
 });
}
```

2.3. RxJS — busca com debounce

Implemente um campo de busca reativo em um componente Angular que:

- Aguarde 500 ms após o usuário parar de digitar antes de disparar a requisição (debounce)
- Cancele a requisição anterior caso o usuário digite novamente (evite race condition)
- Exiba um indicador de loading enquanto a requisição está em andamento

- Gerencie a subscription sem memory leak

Mostre o serviço, o componente e o template com async pipe.

1- Eu escolhi usar signal(false) no lugar de uma propriedade comum porque o estaCarregando é um estado de UI, e Signals foram feitos exatamente para isso. A grande vantagem é que o Angular sabe exatamente onde esse valor é lido no template e atualiza apenas aquele trecho do DOM, sem verificar o componente inteiro. É mais performático e mais intencional, fica claro que aquilo é estado, não um dado derivado de um evento.

2- Para atualizar o Signal dentro do fluxo RxJS, usei .set() dentro do tap e do finalize. O tap 'espia' o fluxo e liga o loading antes da requisição disparar, sem interferir nos dados. Já o finalize garante que o loading seja desligado sempre, seja a requisição um sucesso ou um erro. É a nossa apólice de seguro: o spinner jamais ficará travado na tela.

3- O passo final foi usar o toSignal() para transformar o Observable de resultados em um Signal. Isso elimina o async pipe do template e nos dá um código mais limpo e homogêneo: o template passa a ler dois Signals, sem precisar entender o que é Observable ou o que é dado comum. O Angular cuida da inscrição internamente e cancela tudo automaticamente quando o componente é destruído. É a ponte perfeita entre o mundo do RxJS e o mundo dos Signals.

```
// O Componente
import { Component, OnInit, signal, inject } from '@angular/core';
import { FormControl } from '@angular/forms';
import { Observable, debounceTime, distinctUntilChanged, switchMap, tap, finalize, of } from 'rxjs';

@Component({ ... })
export class BuscaPessoasComponent implements OnInit {
 estaCarregando = signal(false);
  searchControl = new FormControl('');
 resultados$!: Observable<any[]>;
 private pessoaService = inject(PessoaService);
 ngOnInit(): void {
   this.resultados$ = this.searchControl.valueChanges.pipe(
     debounceTime(500),
     distinctUntilChanged(),
     tap(() => this.estaCarregando.set(true)),
     switchMap(termo => {
       if (!termo?.trim()) {
          this.estaCarregando.set(false);
          return of([]);
       }
       return this.pessoaService.buscar(termo).pipe(
         finalize(() => this.estaCarregando.set(false))
       );
     })
   );
 }
}

// O Template
<div *ngIf="estaCarregando()">Carregando resultados...</div>
<ul *ngIf="!estaCarregando()">
 <li *ngFor="let pessoa of resultados$ | async">
   {{ pessoa.nome }}
 </li>
</ul>

// O Serviço
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, delay } from 'rxjs';
@Injectable({ providedIn: 'root' })
export class PessoaService {
```

```
 constructor(private http: HttpClient) {}
 buscar(termo: string): Observable<any[]> {
   return this.http.get<any[]>(`https://api.exemplo.com/pessoas?q=${termo}`);
 }
}
```

2.4. Performance — OnPush e trackBy

Considere uma lista com centenas de itens renderizados com @for (ngFor). Explique:

- Por que usar trackBy melhora a performance e como implementá-lo corretamente

Por padrão, quando a lista de itens é reatribuída (ex: chegou uma nova resposta da API), o Angular não sabe o que mudou e o que não mudou. Ele joga todos os elementos do DOM fora e recria do zero, mesmo que 99% dos itens sejam idênticos.
O trackBy é uma função que diz ao Angular: "Use este identificador único para reconhecer cada item. Se o id for o mesmo, o elemento já existe no DOM, não recrie."

```
// No componente
trackByPessoaId(index: number, pessoa: Pessoa): number {
 return pessoa.id;
}

<!-- No template -->
@for (pessoa of pessoas; track pessoa.id) {
 <app-pessoa-card [pessoa]="pessoa" />
}
<!-- Sintaxe legada com trackBy -->
<li *ngFor="let pessoa of pessoas; trackBy: trackByPessoaId">
```

- Como ChangeDetectionStrategy.OnPush pode reduzir ciclos desnecessários de detecção neste cenário

Se o usuário clica em um botão no componente PAI e nenhum @Input() dos filhos mudou, o Angular pula todos os 500 PessoaCardComponent na verificação. O ganho de performance é grande.

```
import { Component, Input, ChangeDetectionStrategy } from '@angular/core';

@Component({
 selector: 'app-pessoa-card',
 template: `<p>{{ pessoa.nome }}</p>`,
 changeDetection: ChangeDetectionStrategy.OnPush
})
export class PessoaCardComponent {
 @Input() pessoa!: Pessoa;
}
```

- Qual seria o impacto de usar a estratégia Default neste caso

Com ChangeDetectionStrategy.Default, o Angular usa uma abordagem pessimista: a cada evento disparado, seja um clique, uma resposta HTTP ou até um setInterval, ele percorre e verifica todos os componentes da árvore, do pai ao filho mais profundo.
Em uma lista de 500 itens, isso significa que, quando o usuário digita no campo de busca, o Angular verifica os 500 componentes filhos, mesmo que nenhum @Input() deles tenha mudado.

3. Gerenciamento de Estado

3.1. Angular Signals — estado local

Crie um componente de contador de itens no carrinho usando exclusivamente Signals. O componente deve expor:

- Um signal para a lista de itens

- Um computed para o total (quantidade × preço)
- Um método para adicionar e remover itens
- Um output() que emite sempre que o total mudar

```
import { Component, computed, signal, output, effect } from '@angular/core';

export interface ItemCarrinho {
 id: number;
 nome: string;
 preco: number;
 quantidade: number;
}

@Component({
 selector: 'app-carrinho',
 standalone: true,
 templateUrl: './carrinho.component.html'
})
export class CarrinhoComponent {

 itens = signal<ItemCarrinho[]>([]);

 total = computed(() =>
   this.itens().reduce((acc, item) => acc + item.preco * item.quantidade, 0)
 );

 totalMudou = output<number>();

 constructor() {
   effect(() => {
     this.totalMudou.emit(this.total());
   });
 }

 adicionarItem(novoItem: Omit<ItemCarrinho, 'quantidade'>): void {
   this.itens.update(lista => {
     const existente = lista.find(i => i.id === novoItem.id);
     if (existente) {
       return lista.map(i =>
         i.id === novoItem.id ? { ...i, quantidade: i.quantidade + 1 } : i
       );
     }
     return [...lista, { ...novoItem, quantidade: 1 }];
   });
 }

 removerItem(id: number): void {
   this.itens.update(lista =>
     lista
       .map(i => i.id === id ? { ...i, quantidade: i.quantidade - 1 } : i)
       .filter(i => i.quantidade > 0)
   );
 }
}

<div>
 <h2>Carrinho</h2>

 @if (itens().length === 0) {
   <p>Seu carrinho está vazio.</p>
 } @else {
   @for (item of itens(); track item.id) {
     <div>
       <span>{{ item.nome }} — R$ {{ item.preco }} × {{ item.quantidade }}</span>
       <button (click)="adicionarItem(item)">+</button>
       <button (click)="removerItem(item.id)">-</button>
```

```
    </div>
   }
 }

 <strong>Total: R$ {{ total() | number: '1.2-2' }}</strong>
</div>
```

3.2. Gerenciamento de Estado com NgRx (Feature To-do)

Implemente a estrutura de estado para uma lista de tarefas (To-do) utilizando os padrões recomendados do NgRx. A implementação deve incluir:

- **Actions:** Definir ações para: `loadTodos`, `loadTodosSuccess`, `loadTodosError` e `toggleTodoComplete`.
- **Reducer:** Implementar o estado inicial e a transição de estados usando `createReducer`, garantindo a tipagem forte do estado.
- **Selectors:** Criar seletores utilizando `createSelector`:
    - `selectAllTodos`: Retorna a lista completa.
    - `selectPendingTodos`: Retorna apenas as tarefas não concluídas.
- **Effect:** Criar um `Effect` que gerencie o fluxo assíncrono: ao disparar `loadTodos`, deve realizar uma chamada HTTP (mockada) e despachar a ação de sucesso ou erro conforme o resultado.

Não é necessário implementar o back-end — use HttpClient com uma URL fictícia.

```
// Model
export interface Todo {
 id: number;
 titulo: string;
 concluido: boolean;
}

// Actions
import { createActionGroup, emptyProps, props } from '@ngrx/store';
import { Todo } from './todo.model';

export const TodoActions = createActionGroup({
 source: 'Todo',
 events: {
   'Load Todos': emptyProps(),
   'Load Todos Success': props<{ todos: Todo[] }>(),
   'Load Todos Error': props<{ erro: string }>(),
   'Toggle Todo Complete': props<{ id: number }>(),
 }
});
import { createFeature, createReducer, on } from '@ngrx/store';
import { Todo } from './todo.model';
import { TodoActions } from './todo.actions';

// State + Reducer
export interface TodoState {
 todos: Todo[];
 carregando: boolean;
 erro: string | null;
}

const estadoInicial: TodoState = {
 todos: [],
 carregando: false,
 erro: null,
};

export const todoFeature = createFeature({
 name: 'todo',
```

```
 reducer: createReducer(
   estadoInicial,

   on(TodoActions.loadTodos, state => ({
     ...state,
     carregando: true,
     erro: null,
   })),

   on(TodoActions.loadTodosSuccess, (state, { todos }) => ({
     ...state,
     todos,
     carregando: false,
   })),

   on(TodoActions.loadTodosError, (state, { erro }) => ({
     ...state,
     erro,
     carregando: false,
   })),

   on(TodoActions.toggleTodoComplete, (state, { id }) => ({
     ...state,
     todos: state.todos.map(todo =>
       todo.id === id ? { ...todo, concluido: !todo.concluido } : todo
     ),
   }))
 )
});
export const { selectTodoState, selectTodos, selectCarregando } = todoFeature;

// Selectors
import { createSelector } from '@ngrx/store';
import { selectTodos } from './todo.reducer';

export const selectAllTodos = selectTodos;

export const selectPendingTodos = createSelector(
 selectAllTodos,
 todos => todos.filter(todo => !todo.concluido)
);

// Service
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Todo } from './todo.model';

@Injectable({ providedIn: 'root' })
export class TodoService {
 private apiUrl = 'https://api.exemplo.com/todos';

 constructor(private http: HttpClient) {}

 buscarTodos(): Observable<Todo[]> {
   return this.http.get<Todo[]>(this.apiUrl);
 }
}

// Effects
import { Injectable } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { catchError, map, switchMap } from 'rxjs/operators';
import { of } from 'rxjs';
import { TodoActions } from './todo.actions';
import { TodoService } from './todo.service';
```

```
@Injectable()
export class TodoEffects {
 constructor(
   private actions$: Actions,
   private todoService: TodoService
 ) {}

 loadTodos$ = createEffect(() =>
   this.actions$.pipe(
     ofType(TodoActions.loadTodos),
     switchMap(() =>
       this.todoService.buscarTodos().pipe(
         map(todos => TodoActions.loadTodosSuccess({ todos })),
         catchError(err => of(TodoActions.loadTodosError({ erro: err.message })))
       )
     )
   )
 );
}

// Configuração
import { provideStore } from '@ngrx/store';
import { provideEffects } from '@ngrx/effects';
import { todoFeature } from './todo.reducer';
import { TodoEffects } from './todo.effects';

export const appConfig = {
 providers: [
   provideStore({ [todoFeature.name]: todoFeature.reducer }),
   provideEffects(TodoEffects),
 ]
};

// Uso no Componente
@Component({ standalone: true, ... })
export class TodoListComponent {
 private store = inject(Store);

 todos$ = this.store.select(selectAllTodos);
 pendentes$ = this.store.select(selectPendingTodos);
 carregando$ = this.store.select(selectCarregando);

 ngOnInit() {
   this.store.dispatch(TodoActions.loadTodos());
 }

 alternarConclusao(id: number) {
   this.store.dispatch(TodoActions.toggleTodoComplete({ id }));
 }
}
```

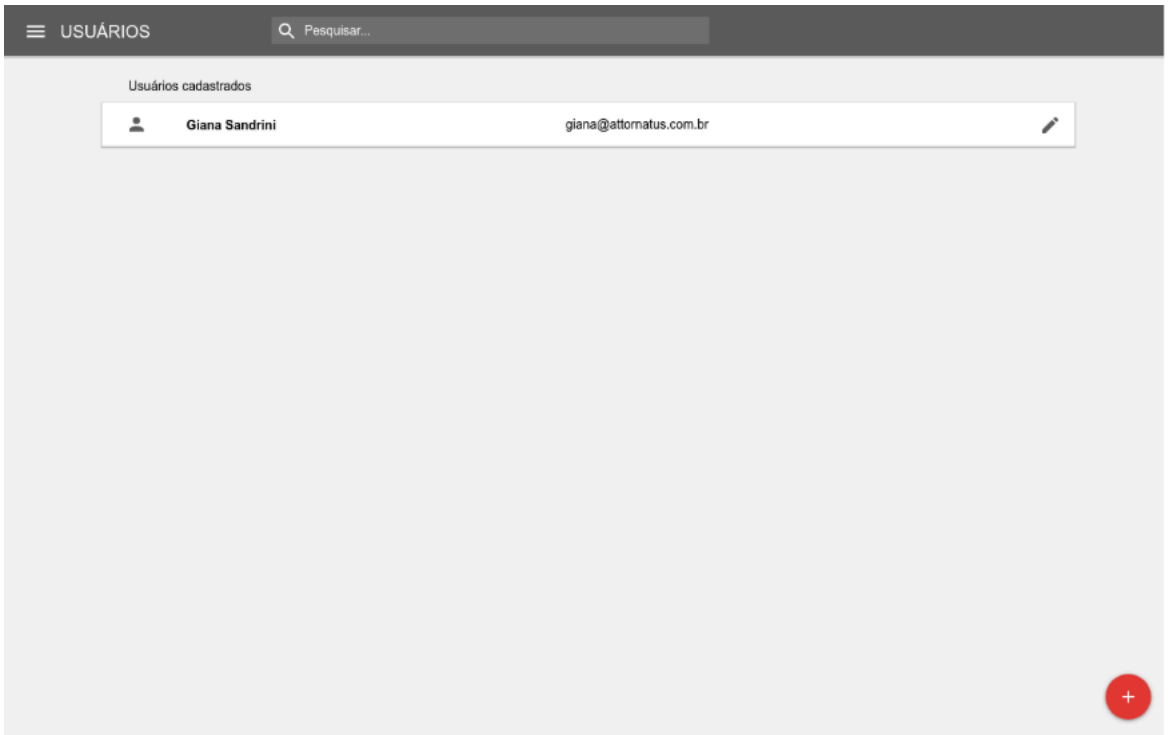
4. Desafio Prático — Aplicação Angular

Stack obrigatória: Angular 17+ · Angular Material · NgRx ou Signals · RxJS · Vitest ou Jest

4.1 O que construir

Reproduzir o protótipo de uma listagem de usuários. A listagem deverá ter as seguintes funcionalidades:

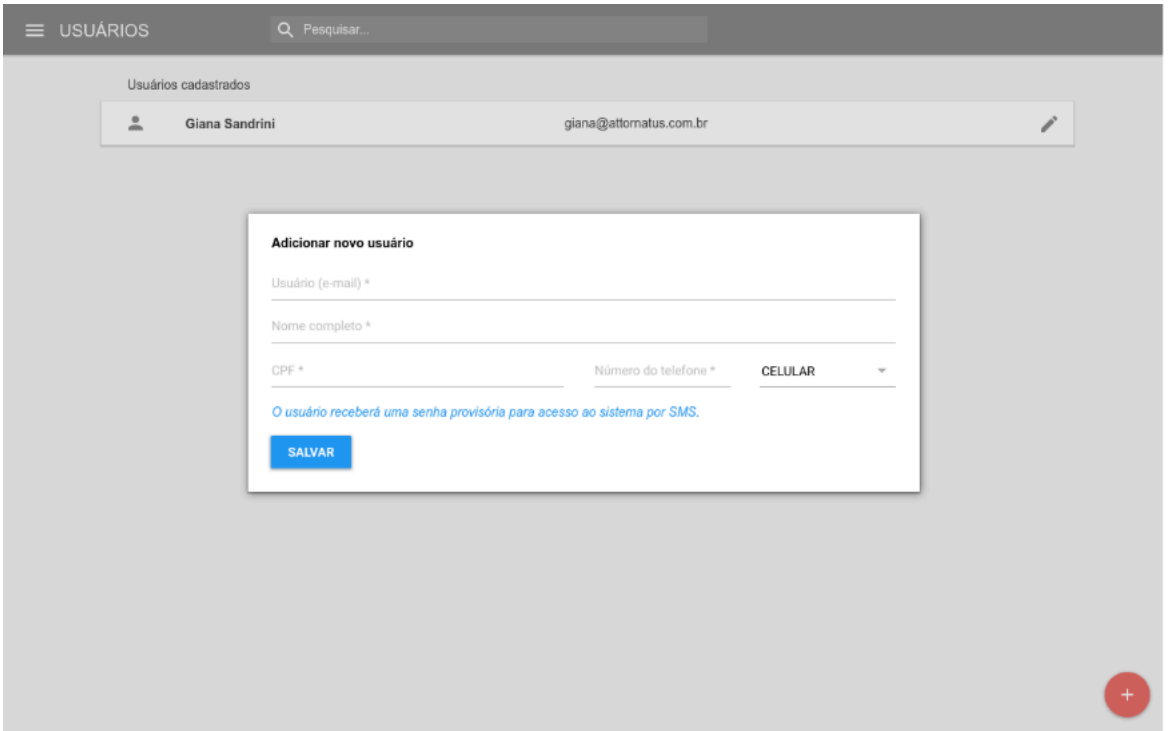
Tela de listagem de usuários:



Listagem de usuários:

- Cards com os campos: nome, e-mail e botão de editar
- Filtro por nome com debounce de 300ms
- Estado de loading durante o carregamento e mensagem de erro em caso de falha
- Os dados podem vir de uma API mockada (JSON Server, MSW ou array estático em serviço)

Modal de cadastro de novo usuário com abertura através do botão vermelho que aparece na listagem:



Criação e edição de usuário (em modal):

- Formulário reativo com campos: e-mail (obrigatório), nome (obrigatório), cpf (obrigatório), telefone (obrigatório) e tipo de telefone.
- Validação com mensagens de erro por campo
- Botão de salvar desabilitado enquanto o formulário estiver inválido
- Quando for edição o formulário deve ser preenchido automaticamente

4.2 Requisitos técnicos

- Pelo menos 2 operadores RxJS além de map e tap em uso real (ex: switchMap, forkJoin, catchError, debounceTime)
- Componentes standalone
- Subscriptions gerenciadas sem memory leaks (takeUntilDestroyed, take, async pipe ou unsubscribe manual no ngOnDestroy)
- Cobertura de testes acima de 60% (Preferencialmente usando Vitest ou Jest)

4.3 Diferenciais (não obrigatórios)

- Nx Monorepo com separação em bibliotecas (ex: feature-users, data-access-users, ui)
- Paginação na listagem
- Validação de formato dos dados do formulário (e-mail,cpf e telefone)
- Melhorias no projeto em relação a tela apresentada no protótipo

---

⚠ Inclua um README.md com instruções de instalação e execução. Projetos que não rodam localmente não serão avaliados.

✅ Dúvidas? Entre em contato pelo e-mail informado no processo seletivo. Boa sorte! 🚀

Estamos ansiosos para conhecer a sua solução.